Attention Is All You Need

- NeurIPS 17 -

Kookmin Univ.
Se Won Hong

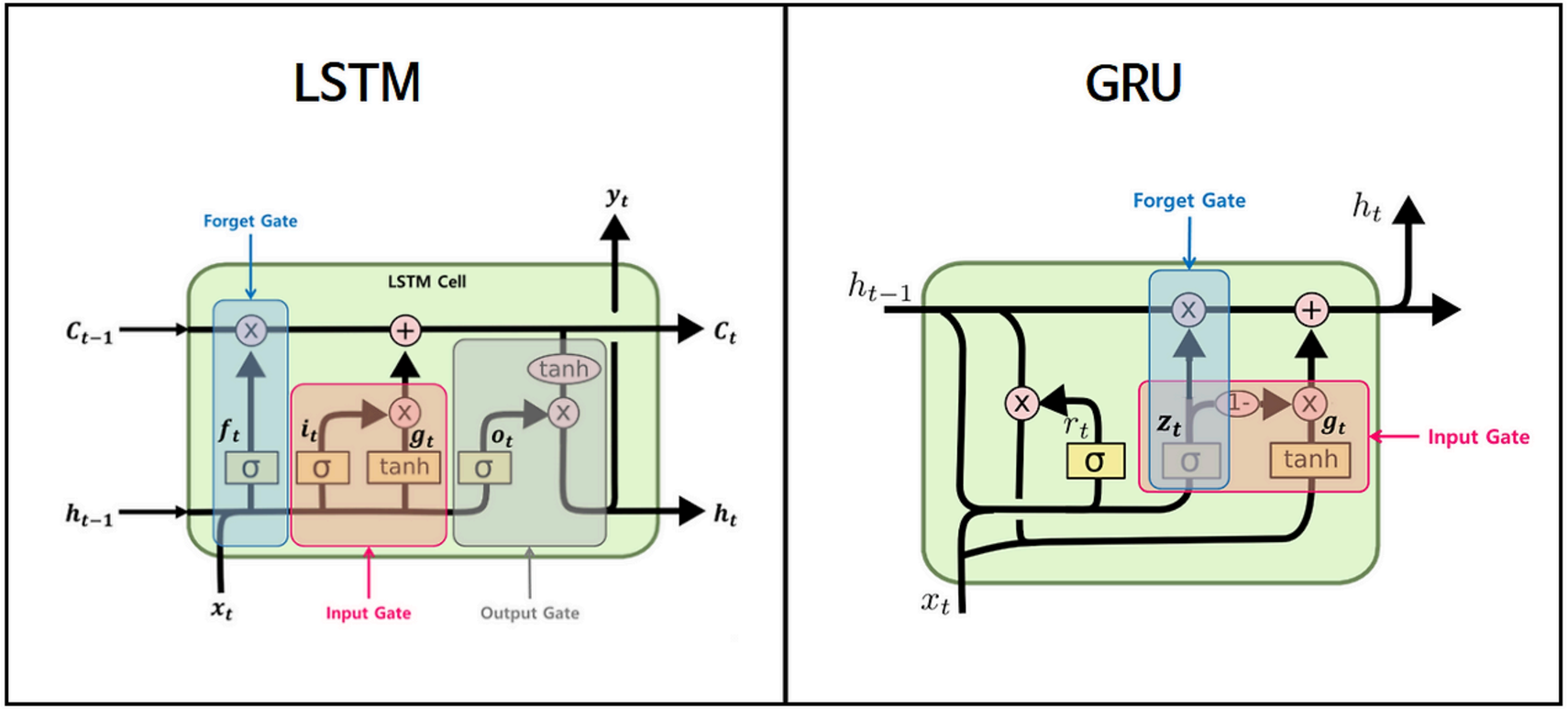
V. Vaswani et al., "Attention Is All You Need," in Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, USA, 2017,

Index

Introduction

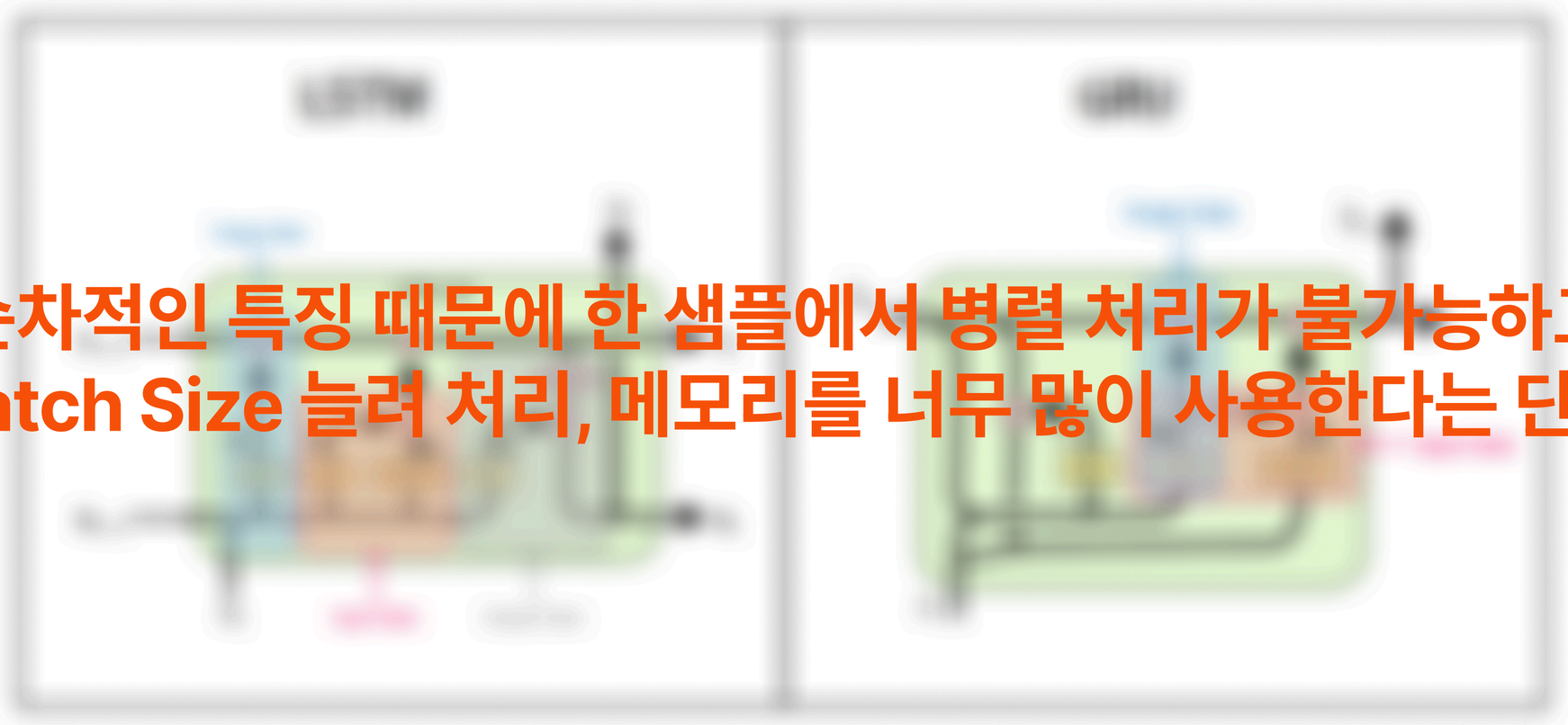
Introduction

이전 연구(Recurrent Models)



Introduction

이전 연구(Recurrent Models)

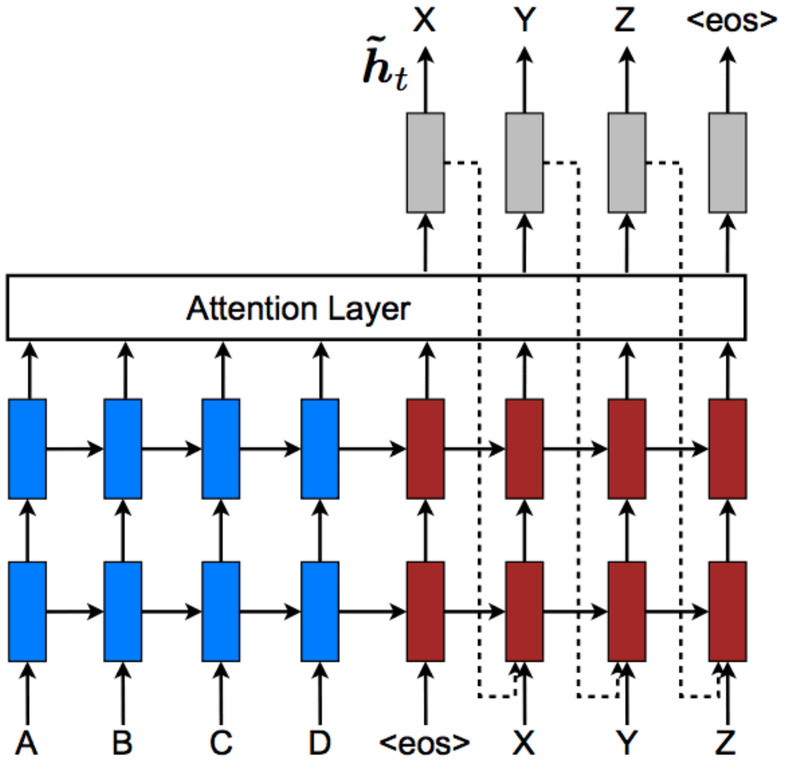


순차적인 특징 때문에 한 샘플에서 병렬 처리가 불가능하고
Batch Size 늘려 처리, 메모리를 너무 많이 사용한다는 단점

Introduction

이전 연구(Recurrent Models) + Attention mechanism

- 성능 향상을 위한 보조 구성 요소로서의 어텐션 메커니즘
- 순차 연산에 기반한 병렬 처리의 한계

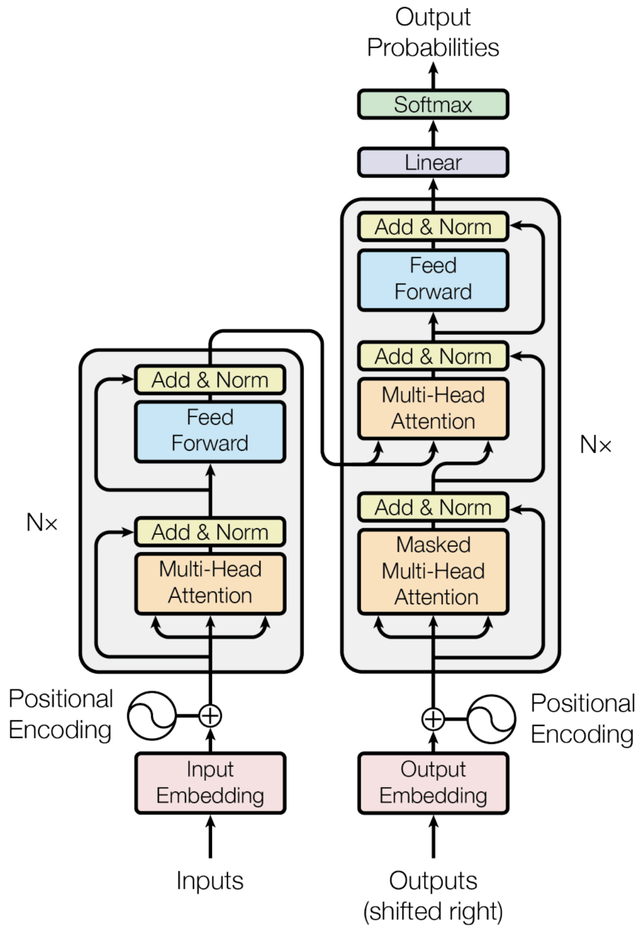


Model

Model

Transformer

- 트랜스포머 : 인코더 구조 + 디코더 구조
- 인코더 : 6개 레이어
- 디코더 : 6개 레이어



Model

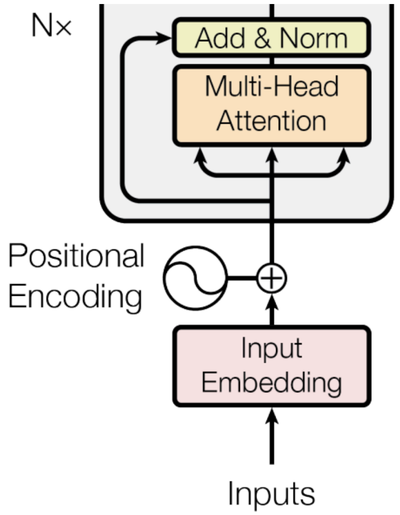
Encoder

First Layer : Multi-Head Self-Attention

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**

Second Layer : position-wise FFNN

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**



Model

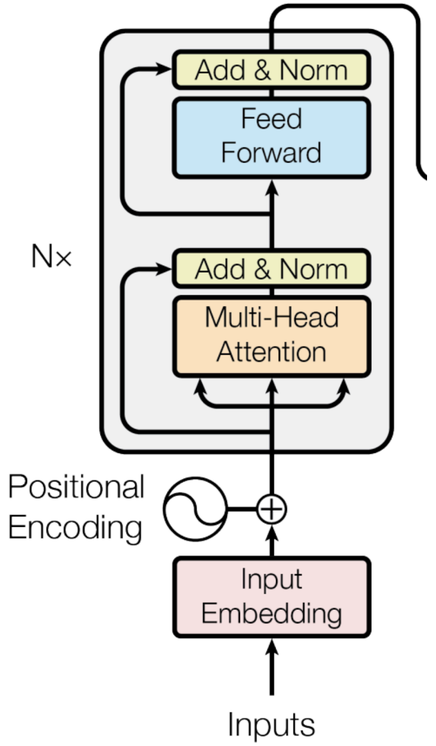
Encoder

First Layer : Multi-Head Self-Attention

- Self connection, LayerNorm
- LayerNorm(x + Sublayer(x))

Second Layer : position-wise FFNN

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**



Model

Encoder

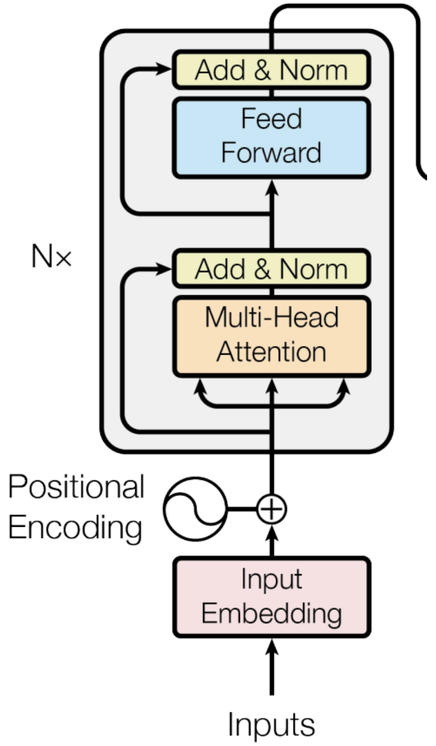
First Layer : Multi-Head Self-Attention

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**

Second Layer : position-wise FFNN

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**

X N



Model

Decoder

First Layer : Masked Multi-Head Self-Attention

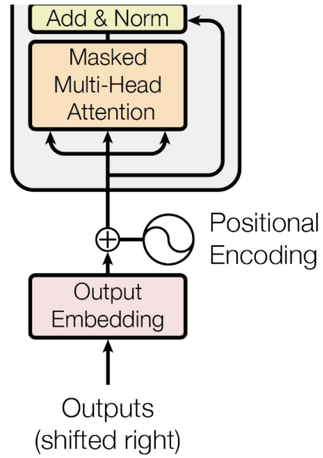
- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**

Second Layer : Encoder - Decoder Cross Attention

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x)**

Thired Layer : position-wise FFNN

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**



Model

Decoder

First Layer : Masked Multi-Head Self-Attention

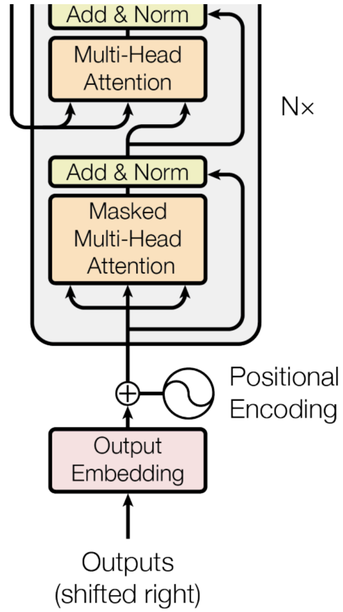
- Self connection, LayerNorm
- LayerNorm(x + Sublayer(x))

Second Layer : Encoder - Decoder Cross Attention

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x)**

Thired Layer : position-wise FFNN

- Self connection, LayerNorm
- LayerNorm(x + Sublayer(x))



Model

Decoder

First Layer : Masked Multi-Head Self-Attention

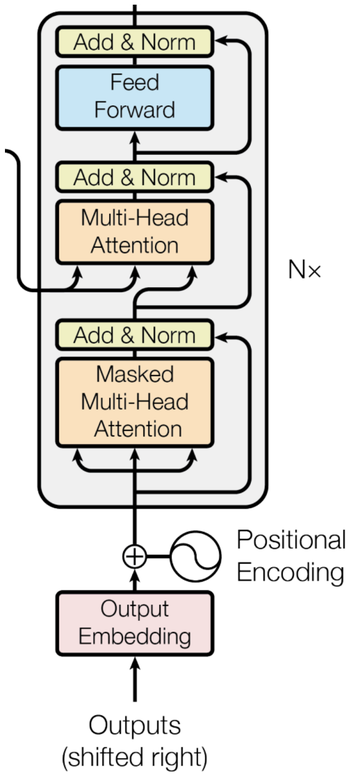
- Self connection, LayerNorm
- LayerNorm(x + Sublayer(x))

Second Layer : Encoder - Decoder Cross Attention

- Self connection, LayerNorm
- LayerNorm(x + Sublayer(x)

Thired Layer : position-wise FFNN

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**



Model

Decoder

First Layer : Masked Multi-Head Self-Attention

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**

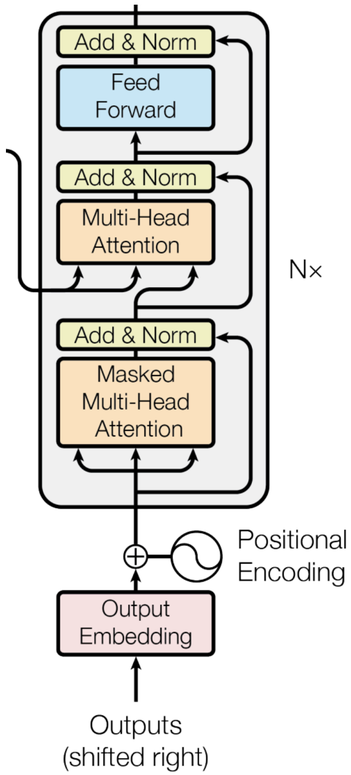
Second Layer : Encoder - Decoder Cross Attention

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x)**

Thired Layer : position-wise FFNN

- **Self connection, LayerNorm**
- **LayerNorm(x + Sublayer(x))**

X N



Model

Scaled Dot-Product Attention

X : n x d

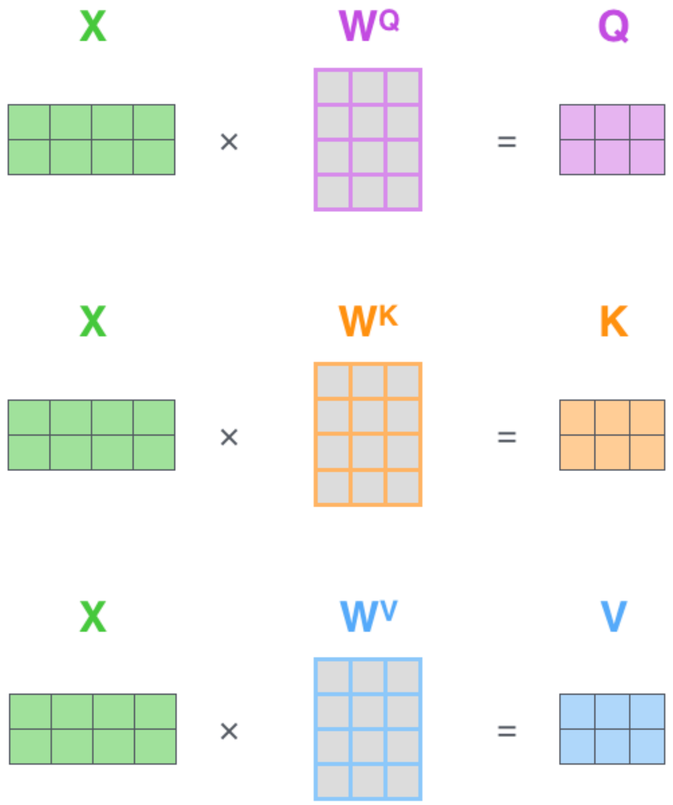
W_q,k : d x d_k

W_v : d x d_v

Q = X x W_q

K = X x W_k

V = X x W_v



Model

Scaled Dot-Product Attention

Q = n x d_k

K = n x d_k

V = n x d_v

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Model

Scaled Dot-Product Attention

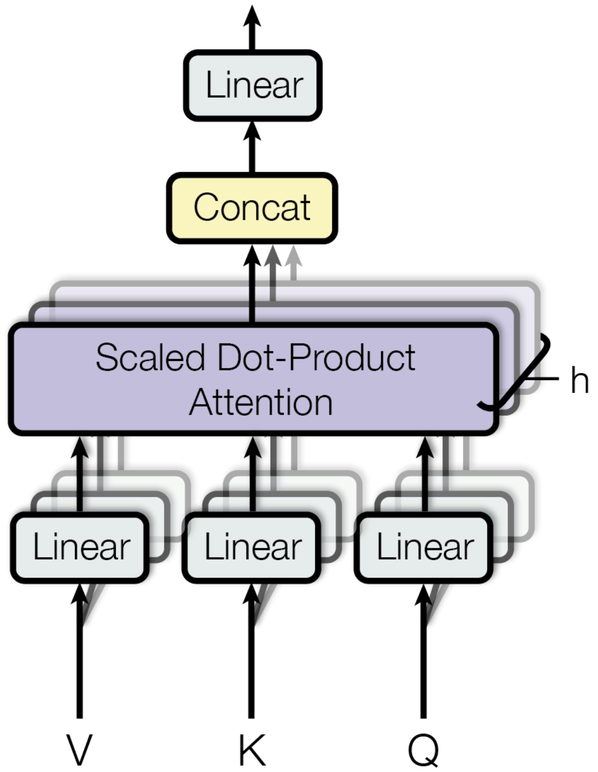
- d_k값이 커질 수록 QK^T값이 매우 커져 기울기 값이 0으로 수렴하게 됨
- 이러한 현상을 방지하기 위해 √d_k를 통해 크기 조절

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Model

Multi-head Attention

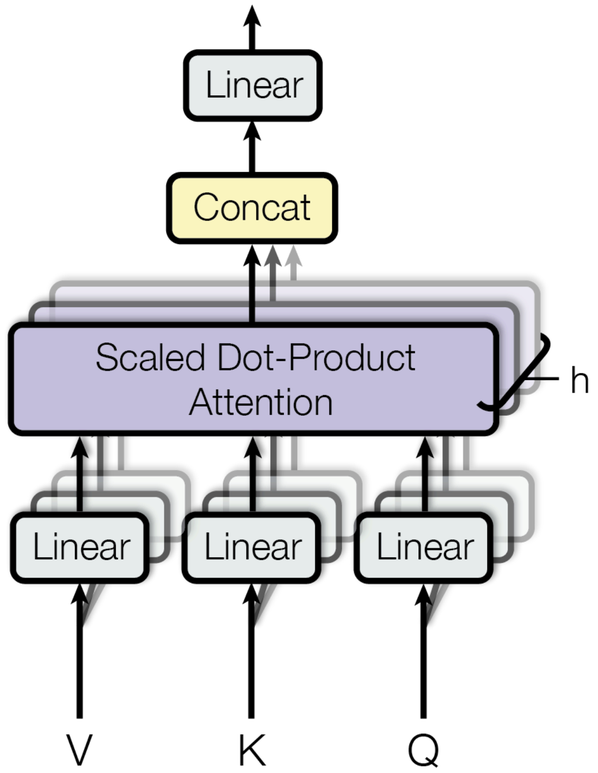
- 입력 시퀀스로부터 Q, K, V 행렬 생성
  - (seq_len × d_model)
- Q, K, V를 각각 h개의 head로 분할 및 선형 투영
  - Q, K: seq_len × {(d_k)/h}, V: seq_len × {(d_v)/h}
- 각 head에서 어텐션 연산 수행
  - (seq_len × (d_v)/h) × h개
- h개의 어텐션 출력을 concat
  - seq_len × d_v
- concat된 출력에 W_o를 곱해 최종 출력 생성
  - 선형 변환을 통한 출력 조정 (크기: (seq_len × d_model))



Model

Multi-head Attention

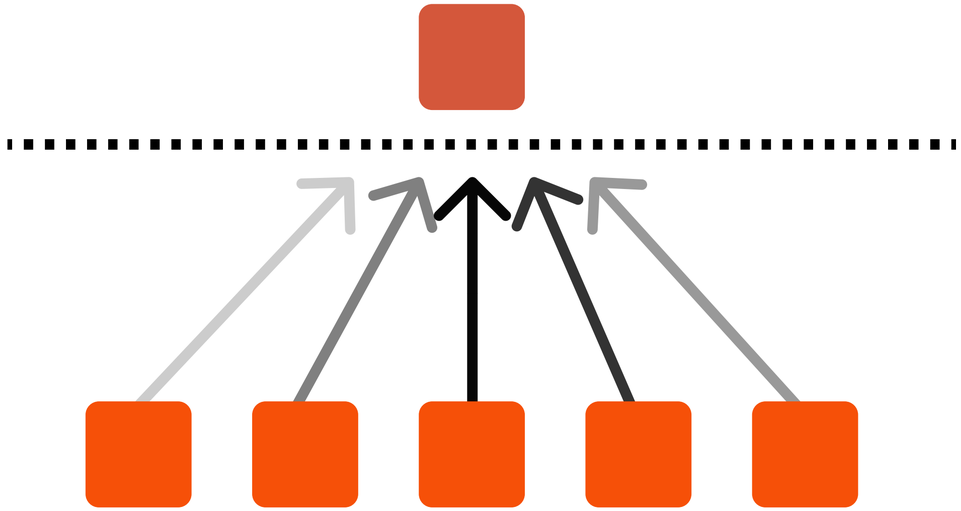
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



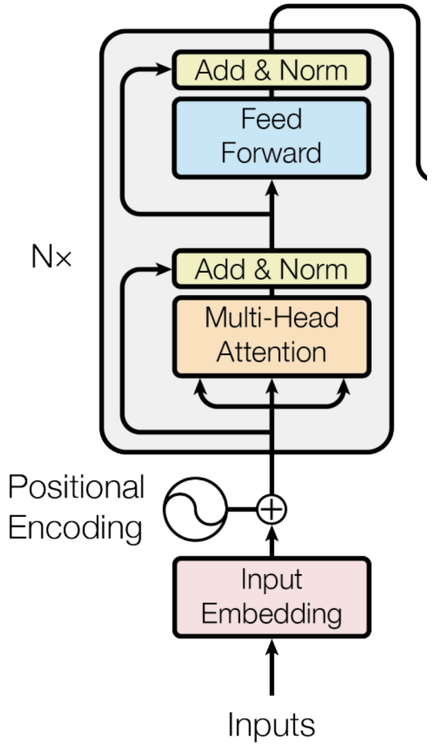
Model

Multi-head Attention - Encoder

Encoder에서의 Q, K, V는 이전 레이어 출력으로부터 생성된 벡터



Self-Attention

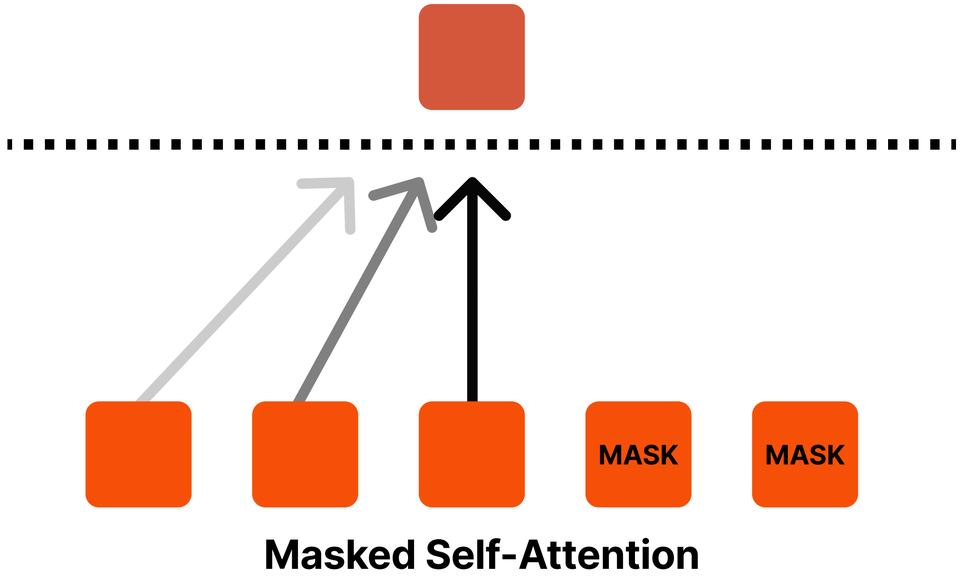


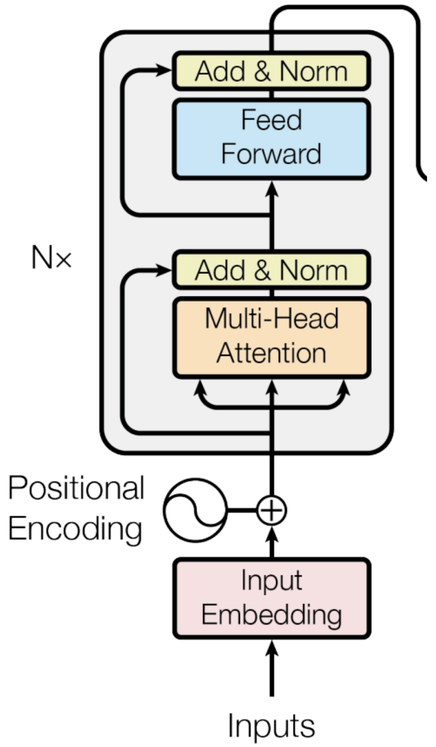
Model

Multi-head Attention - Decoder

Decoder에서의 Q, K, V는 이전 레이어 출력으로부터 생성된 벡터

Decoder는 미래 정보를 MASK하여 정보 흐름을 막음

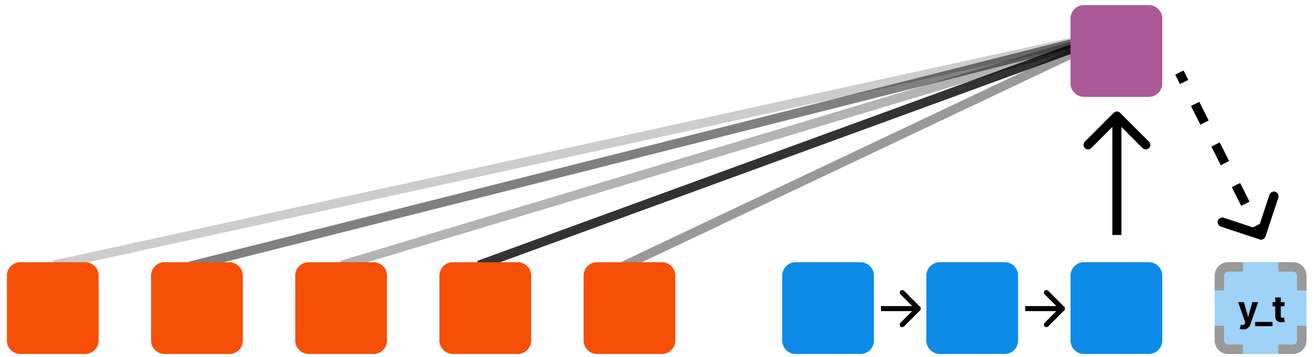




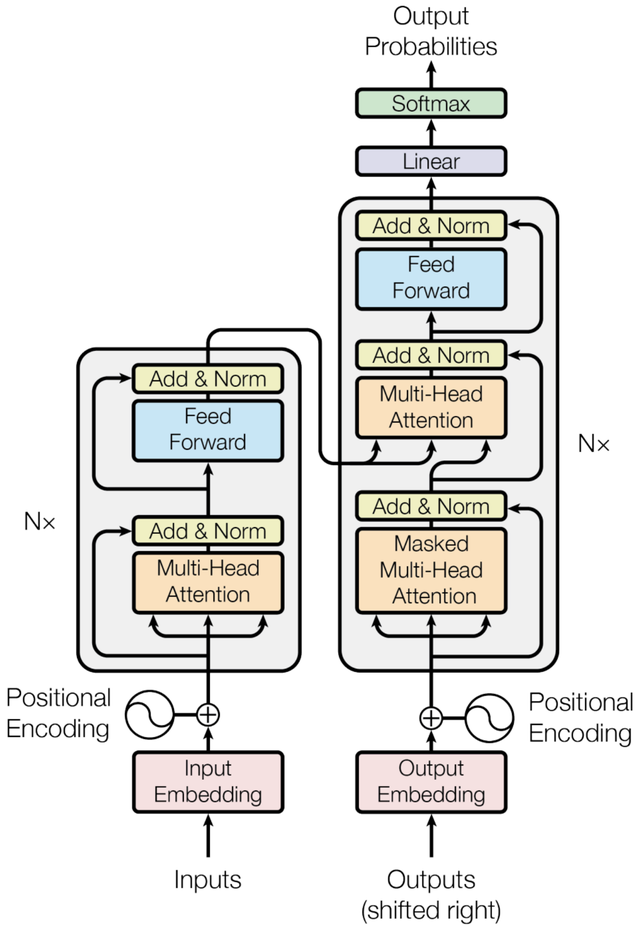
Model

Multi-head Attention - Encoder / Decoder

Encoder/Decoder 경우 Q는 Decoder에서,
K와 V는 Encoder 이전 레이어 출력으로부터 생성된 벡터



Cross-Attention



Model

Position-wise Feed-Forward Networks

- Encoder와 Decoder 각 레이어는 위치별 동일하게 적용되는 FFNN 포함
- 두 개의 선형 변환과 ReLU로 구성된 비선형 변환 구조
- 레이어별로 다른 파라미터를 사용하는 d_model 512, d_ff 2048 구조

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

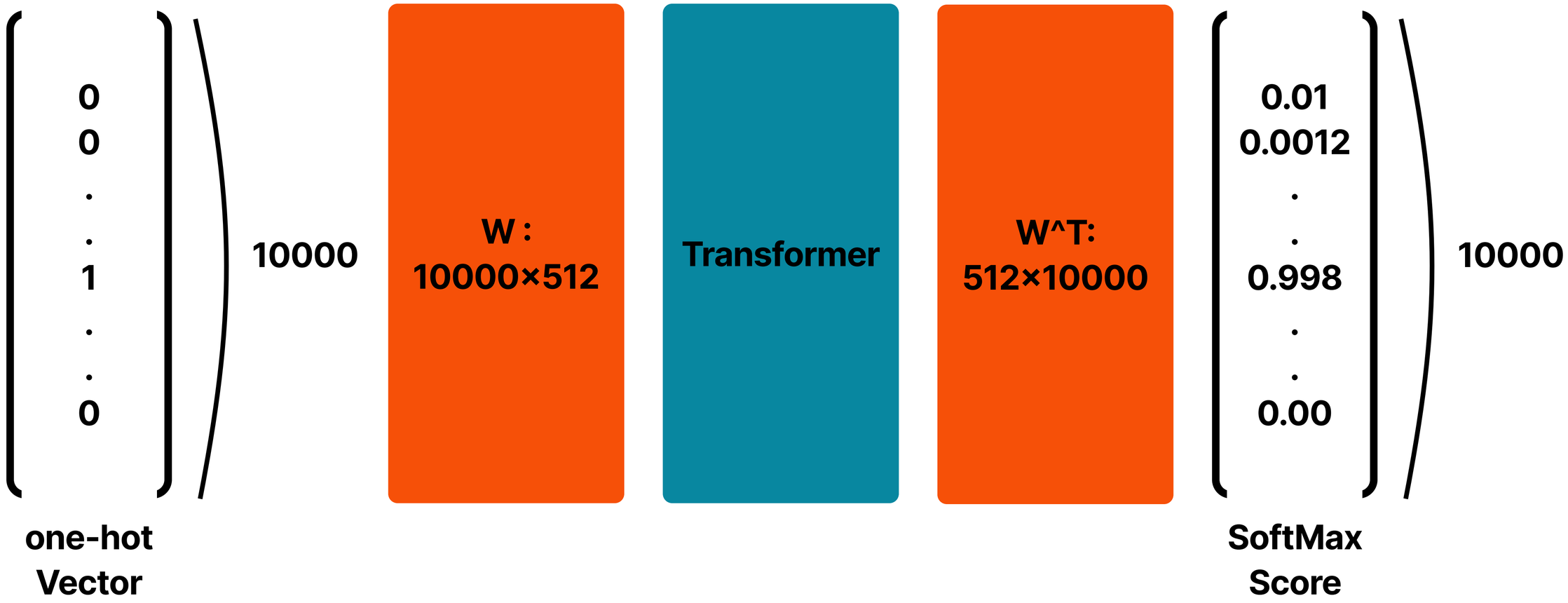
Model

Embeddings and Softmax

- 입력과 출력 토큰을 d_model 차원의 벡터로 변환하기 위해 학습된 W을 사용
- Decoder 출력은 선형 변환과 소프트맥스를 통해 다음 토큰의 확률 분포로 변환됨
- Decoder 출력의 선형 변환에 사용되는 W는 입력 W행렬과 가중치를 공유함
- 스케일 조정을 위해 입력 임베딩 벡터에 √d_model를 곱함

Model

Embeddings and Softmax



Model

Positional Encoding

- Transformer에 들어가는 Sequence에는 위치 정보가 반영되어 있지 않음
- 이를 반영하기 위해 Positional Encoding을 사용함

- pos : sequence 위치
- i : 차원 인덱스
- d_model : sequence 임베딩 차원

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

Model

Why Self-Attention

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n : 시퀀스 길이
d : 임베딩 사이즈
k : 커널 사이즈
r : 주변 이웃 사이즈

Experiment

Experiment

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	